Notes on structured concurrency, or: Go statement considered harmful

Wed 25 April 2018

Every concurrency API needs a way to run code concurrently. Here's some examples of what that looks like using different APIs:

```
go myfunc();                                // Golang

pthread_create(&thread_id, NULL, &myfunc);  /* C with POSIX threads */

spawn(modulename, myfuncname, [])           % Erlang

threading.Thread(target=myfunc).start()     # Python with threads

asyncio.create_task(myfunc())               # Python with asyncio
```

There are lots of variations in the notation and terminology, but the semantics are the same: these all arrange for `myfunc` to start running concurrently to the rest of the program, and then return immediately so that the parent can do other things.

Another option is to use callbacks:

```
QObject::connect(&emitter, SIGNAL(event()),        // C++ with Qt
                 &receiver, SLOT(myfunc()))

g_signal_connect(emitter, "event", myfunc, NULL)   /* C with GObject */

document.getElementById("myid").onclick = myfunc;  // Javascript

promise.then(myfunc, errorhandler)                 // Javascript with Promises

deferred.addCallback(myfunc)                       # Python with Twisted

future.add_done_callback(myfunc)                   # Python with asyncio
```

Again, the notation varies, but these all accomplish the same thing: they arrange that from now on, if and when a certain event occurs, then `myfunc` will run. Then once they've set that up, they immediately return so the caller can do other things. (Sometimes callbacks get dressed up with fancy helpers like promise combinators, or Twisted-style protocols/transports, but the core idea is the same.)

And... that's it. Take any real-world, general-purpose concurrency API, and you'll probably find that it falls into one or the other of those buckets (or sometimes both, like asyncio).

But my new library Trio is weird. It doesn't use either approach. Instead, if we want to run `myfunc` and `anotherfunc` concurrently, we write something like:

```
async with trio.open_nursery() as nursery:
    nursery.start_soon(myfunc)
    nursery.start_soon(anotherfunc)
```

When people first encounter this "nursery" construct, they tend to find it confusing. Why is there an indented block? What's this `nursery` object, and why do I need one before I can spawn a task? Then they realize that it prevents them from using patterns they've gotten used to in other frameworks, and they get really annoyed. It feels quirky and idiosyncratic and too high-level to be a basic primitive. These are understandable reactions! But bear with me.

In this post, I want to convince you that nurseries aren't quirky or idiosyncratic at all, but rather a new control flow primitive that's just as fundamental as for loops or function calls. And furthermore, the other approaches we saw above – thread spawning and callback registration – should be removed entirely and replaced with nurseries.

Sound unlikely? Something similar has actually happened before: the `goto` statement was once the king of control flow. Now it's a punchline. A few languages still have something they call `goto`, but it's different and far weaker than the

original `goto`. And most languages don't even have that. What happened? This was so long ago that most people aren't familiar with the story anymore, but it turns out to be surprisingly relevant. So we'll start by reminding ourselves what a `goto` was, exactly, and then see what it can teach us about concurrency APIs.

Contents:

- What is a `goto` statement anyway?
- What is a `go` statement anyway?
- What happened to `goto`?
  - `goto`: the destroyer of abstraction
  - A surprise benefit: removing `goto` statements enables new features
  - `goto` statements: not even once
- `go` statement considered harmful
  - `go` statements: not even once
- Nurseries: a structured replacement for `go` statements
  - Nurseries preserve the function abstraction.
  - Nurseries support dynamic task spawning.
  - There is an escape.
  - You can define new types that quack like a nursery.
  - No, really, nurseries *always* wait for the tasks inside to exit.
  - Automatic resource cleanup works.
  - Automated error propagation works.
  - A surprise benefit: removing `go` statements enables new features
- Nurseries in practice
- Conclusion
- Comments
- Acknowledgments
- Footnotes

What is a `goto` statement anyway?

Let's review some history: Early computers were programmed using assembly language, or other even more primitive mechanisms. This kinda sucked. So in the 1950s, people like John Backus at IBM and Grace Hopper at Remington Rand started to develop languages like FORTRAN and FLOW-MATIC (better known for its direct successor COBOL).

FLOW-MATIC was very ambitious for its time. You can think of it as Python's great-great-great-...-grandparent: the first language that was designed for humans first, and computers second. Here's some FLOW-MATIC code to give you a taste of what it looked like:

```
 (0) INPUT INVENTORY FILE-A PRICE FILE-B;
     OUTPUT PRICED-INV FILE-C UNPRICED-INV FILE-D;
     HSP D.
 (1) COMPARE PRODUCT-NO(A) WITH PRODUCT-NO(B);
     IF GREATER GO TO OPERATION 10;
     IF EQUAL GO TO OPERATION 5;
     OTHERWISE GO TO OPERATION 2.
 (2) TRANSFER A TO D.
 (3) WRITE-ITEM D.
 (4) JUMP TO OPERATION 8.
 (5) TRANSFER A TO C.
 (6) MOVE UNIT-PRICE(B) TO UNIT-PRICE(C).
 (7) WRITE-ITEM C.
 (8) READ-ITEM A; IF END OF DATA GO TO OPERATION 14.
 (9) JUMP TO OPERATION 1.
(10) READ-ITEM B; IF END OF DATA GO TO OPERATION 12.
(11) JUMP TO OPERATION 1.
(12) SET OPERATION 9 TO GO TO OPERATION 2.
(13) JUMP TO OPERATION 2.
(14) TEST PRODUCT-NO(B) AGAINST ZZZZZZZZZZZZ;
     IF EQUAL GO TO OPERATION 16;
     OTHERWISE GO TO OPERATION 15.
(15) REWIND B.
(16) CLOSE-OUT FILES C, D.
(17) STOP. (END)
```

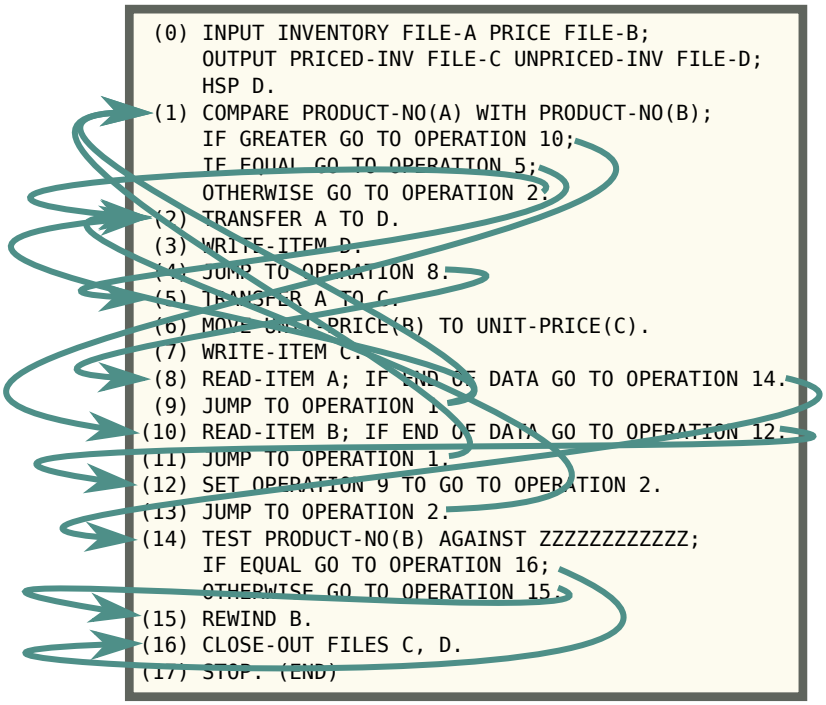
You'll notice that unlike modern languages, there's no `if` blocks, loop blocks, or function calls here – in fact there's no block delimiters or indentation at all. It's just a flat list of statements. That's not because this program happens to be too short to use fancier control syntax – it's because block syntax wasn't invented yet!

Instead, FLOW-MATIC had two options for flow control. Normally, it was sequential, just like you'd expect: start at the top and move downwards, one statement at a time. But if you execute a special statement like `JUMP TO`, then it could directly transfer control somewhere else. For example, statement (13) jumps back to statement (2):

```
 (0) INPUT INVENTORY FILE-A PRICE FILE-B;
     OUTPUT PRICED-INV FILE-C UNPRICED-INV FILE-D;
     HSP D.
 (1) COMPARE PRODUCT-NO(A) WITH PRODUCT-NO(B);
     IF GREATER GO TO OPERATION 10;
     IF EQUAL GO TO OPERATION 5;
     OTHERWISE GO TO OPERATION 2.
 (2) TRANSFER A TO D.
 (3) WRITE-ITEM D.
 (4) JUMP TO OPERATION 8.
 (5) TRANSFER A TO C.
 (6) MOVE UNIT-PRICE(B) TO UNIT-PRICE(C).
 (7) WRITE-ITEM C.
 (8) READ-ITEM A; IF END OF DATA GO TO OPERATION 14.
 (9) JUMP TO OPERATION 1.
(10) READ-ITEM B; IF END OF DATA GO TO OPERATION 12.
(11) JUMP TO OPERATION 1.
(12) SET OPERATION 9 TO GO TO OPERATION 2.
(13) JUMP TO OPERATION 2.
(14) TEST PRODUCT-NO(B) AGAINST ZZZZZZZZZZZZ;
     IF EQUAL GO TO OPERATION 16;
     OTHERWISE GO TO OPERATION 15.
(15) REWIND B.
(16) CLOSE-OUT FILES C, D.
(17) STOP. (END)
```

Just like for our concurrency primitives at the beginning, there was some disagreement about what to call this "do a one-way jump" operation. Here it's `JUMP TO`, but the name that stuck was `goto` (like "go to", get it?), so that's what I'll use here.

Here's the complete set of `goto` jumps in this little program:



If you think this looks confusing, you're not alone! This style of jump-based programming is something that FLOW-MATIC inherited pretty much directly from assembly language. It's powerful, and a good fit to how computer hardware actually works, but it's super confusing to work with directly. That tangle of arrows is why the term "spaghetti code" was invented. Clearly, we needed something better.

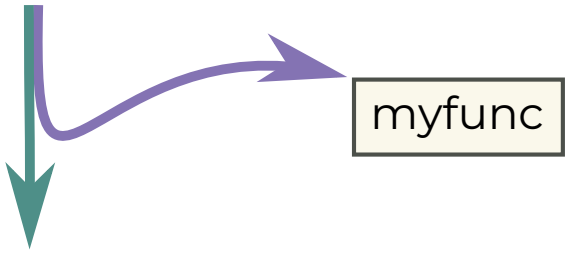
But... what is it about `goto` that causes all these problems? Why are some control structures OK, and some not? How do we pick the good ones? At the time, this was really unclear, and it's hard to fix a problem if you don't understand it.

What is a `go` statement anyway?

But let's hit pause on the history for a moment – everyone knows `goto` was bad. What does this have to do with concurrency? Well, consider Golang's famous `go` statement, used to spawn a new "goroutine" (lightweight thread):

```
// Golang
go myfunc();
```

Can we draw a diagram of its control flow? Well, it's a little different from either of the ones we saw above, because control actually splits. We might draw it like:



Here the colors are intended to indicate that *both* paths are taken. From the perspective of the parent goroutine (green line), control flows sequentially: it comes in the top, and then immediately comes out the bottom. Meanwhile, from the perspective of the child (lavender line), control comes in the top, and then jumps over to the body of `myfunc`. Unlike a

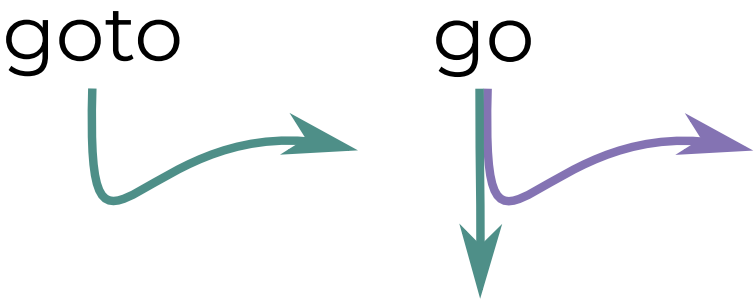
regular function call, this jump is one-way: when running `myfunc` we switch to a whole new stack, and the runtime immediately forgets where we came from.

But this doesn't just apply to Golang. This is the flow control diagram for *all* of the primitives we listed at the beginning of this post:

- Threading libraries usually provide some sort of handle object that lets you `join` the thread later – but this is an independent operation that the language doesn't know anything about. The actual thread spawning primitive has the control flow shown above.
- Registering a callback is semantically equivalent to starting a background thread that (a) blocks until some event occurs, and then (b) runs the callback. (Though obviously the implementation is different.) So in terms of high-level control flow, registering a callback is essentially a `go` statement.
- Futures and promises are the same too: when you call a function and it returns a promise, that means it's scheduled the work to happen in the background, and then given you a handle object to join the work later (if you want). In terms of control flow semantics, this is just like spawning a thread. Then you register callbacks on the promise, so see the previous bullet point.

This same exact pattern shows up in many, many forms: the key similarity is that in all these cases, control flow splits, with one side doing a one-way jump and the other side returning to the caller. Once you know what to look for, you'll start seeing it all over the place – it's a fun game! [1]

Annoyingly, though, there is no standard name for this category of control flow constructs. So just like "`goto` statement" became the umbrella term for all the different `goto`-like constructs, I'm going to use "`go` statement" as a umbrella term for these. Why `go`? One reason is that Golang gives us a particularly pure example of the form. And the other is... well, you've probably guessed where I'm going with all this. Look at these two diagrams. Notice any similarities?



That's right: **go statements are a form of goto statement.**

Concurrent programs are notoriously difficult to write and reason about. So are `goto`-based programs. Is it possible that this might be for some of the same reasons? In modern languages, the problems caused by `goto` are largely solved. If we study how they fixed `goto`, will it teach us how to make more usable concurrency APIs? Let's find out.

What happened to `goto`?

So what is it about `goto` that makes it cause so many problems? In the late 1960s, Edsger W. Dijkstra wrote a pair of now-famous papers that helped make this much clearer: Go to statement considered harmful, and Notes on structured programming (PDF).

`goto`: the destroyer of abstraction

In these papers, Dijkstra was worried about the problem of how you write non-trivial software and get it correct. I can't give them due justice here; there's all kinds of fascinating insights. For example, you may have heard this quote:

> Program testing can be used to show the presence o
> their absence!

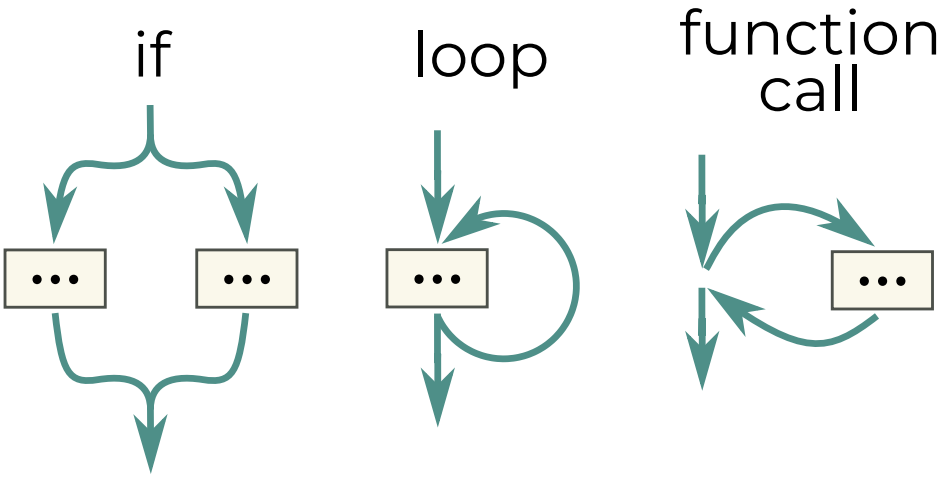
Yep, that's from *Notes on structured programming*. But his major concern was *abstraction*. He wanted to write programs that are too big to hold in your head all at once. To do this, you need to treat parts of the program like a black box – like when you see a Python program do:

```
print("Hello world!")
```

then you don't need to know all the details of how `print` is implemented (string formatting, buffering, cross-platform differences, ...). You just need to know that it will somehow print the text you give it, and then you can spend your energy thinking about whether that's what you want to have happen at this point in your code. Dijkstra wanted languages to support this kind of abstraction.

By this point, block syntax had been invented, and languages like ALGOL had accumulated ~5 distinct types of control structure: they still had sequential flow and `goto`:

And had also acquired variants on if/else, loops, and function calls:



You can implement these higher-level constructs using `goto`, and early on, that's how people thought of them: as a convenient shorthand. But what Dijkstra pointed out is that if you look at these diagrams, there's a big difference between `goto` and the rest. For everything except `goto`, flow control comes in the top → [stuff happens] → flow control comes out the bottom. We might call this the "black box rule": if a control structure has this shape, then in contexts where you don't care about the details of what happens internally, you can ignore the [stuff happens] part, and treat the whole thing as regular sequential flow. And even better, this is also true of any code that's *composed* out of those pieces. When I look at this code:

```
print("Hello world!")
```

I don't have to go read the definition of `print` and all its transitive dependencies just to figure out how the control flow works. Maybe inside `print` there's a loop, and inside the loop there's an if/else, and inside the if/else there's another function call... or maybe it's something else. It doesn't really matter: I know control will flow into `print`, the function will do its thing, and then eventually control will come back to the code I'm reading.

It may seem like this is obvious, but if you have a language with `goto` – a language where functions and everything else are built on top of `goto`, and `goto` can jump anywhere, at any time – then these control structures aren't black boxes at all! If you have a function, and inside the function there's a loop, and inside the loop there's an if/else, and inside the if/else there's a `goto`... then that `goto` could send the control anywhere it wants. Maybe control will suddenly return from another function entirely, one you haven't even called yet. You don't know!

And this breaks abstraction: it means that *every function call is potentially a* `goto` *statement in disguise, and the only way to know is to keep the entire source code of your system in your head at once.* As soon as `goto` is in your language, you stop being able do local reasoning about flow control. That's *why* `goto` leads to spaghetti code.

And now that Dijkstra understood the problem, he was able to solve it. Here's his revolutionary proposal: we should stop thinking of if/loops/function calls as shorthands for `goto`, but rather as fundamental primitives in their own rights – and we should remove `goto` entirely from our languages.

From here in 2018, this seems obvious enough. But have you seen how programmers react when you try to take away their toys because they're not smart enough to use them safely? Yeah, some things never change. In 1969, this proposal was *incredibly controversial*. Donald Knuth defended `goto`. People who had become experts on writing code with `goto` quite reasonably resented having to basically learn how to program again in order to express their ideas using the newer, more constraining constructs. And of course it required building a whole new set of languages.

Left: A traditional `goto`. Right: A domesticated `goto`, as seen in C, C#, Golang, etc. The inability to cross function boundaries means it can still pee on your shoes, but it probably won't rip your face off.

In the end, modern languages are a bit less strict about this than Dijkstra's original formulation. They'll let you break out of multiple nested structures at once using constructs like `break`, `continue`, or `return`. But fundamentally, they're all designed around Dijkstra's idea; even these constructs that push the boundaries do so only in strictly limited ways. In particular, functions – which are the fundamental tool for wrapping up control flow inside a black box – are considered inviolate. You can't `break` out of one function and into another, and a `return` can take you out of the current function, but no further. Whatever control flow shenanigans a function gets up to internally, other functions don't have to care.

This even extends to `goto` itself. You'll find a few languages that still have something they call `goto`, like C, C#, Golang, ... but they've added heavy restrictions. At the very least, they won't let you jump out of one function body and into another. Unless you're working in assembly [2], the classic, unrestricted `goto` is gone. Dijkstra won.

A surprise benefit: removing `goto` statements enables new features

And once `goto` disappeared, something interesting happened: language designers were able to start adding features that depend on control flow being structured.

For example, Python has some nice syntax for resource cleanup: the `with` statement. You can write things like:

```
# Python
with open("my-file") as file_handle:
    ...
```

and it guarantees that the file will be open during the `...` code, but then closed immediately afterward. Most modern languages have some equivalent (RAII, `using`, try-with-resource, `defer`, ...). And they all assume that control flows in an orderly, structured way. If we used `goto` to jump into the middle of our `with` block... what would that even do? Is the file open or not? What if we jumped out again, instead of exiting normally? Would the file get closed? This feature just doesn't work in any coherent way if your language has `goto` in it.

Error handling has a similar problem: when something goes wrong, what should your code do? Often the answer is to pass the buck up the stack to your code's caller, let them figure out how to deal with it. Modern languages have constructs specifically to make this easier, like exceptions, or other forms of automatic error propagation. But your

language can only provide this help if it *has* a stack, and a reliable concept of "caller". Look again at the control-flow spaghetti in our FLOW-MATIC program and imagine that in the middle of that it tried to raise an exception. Where would it even go?

`goto` statements: not even once

So `goto` – the traditional kind that ignores function boundaries – isn't just the regular kind of bad feature, the kind that's hard to use correctly. If it were, it might have survived – lots of bad features have. But it's much worse.

Even if you don't use `goto` yourself, merely having it as an option in your language makes *everything* harder to use. Whenever you start using a third-party library, you can't treat it as a black box – you have to go read through it all to find out which functions are regular functions, and which ones are idiosyncratic flow control constructs in disguise. This is a serious obstacle to local reasoning. And you lose powerful language features like reliable resource cleanup and automatic error propagation. Better to remove `goto` entirely, in favor of control flow constructs that follow the "black box" rule.

`go` statement considered harmful

So that's the history of `goto`. Now, how much of this applies to `go` statements? Well... basically, all of it! The analogy turns out to be shockingly exact.

Go statements break abstraction. Remember how we said that if our language allows `goto`, then any function might be a `goto` in disguise? In most concurrency frameworks, `go` statements cause the exact same problem: whenever you call a function, it might or might not spawn some background task. The function seemed to return, but is it still running in the background? There's no way to know without reading all its source code, transitively. When will it finish? Hard to say. If you have `go` statements, then functions are no longer black boxes with respect to control flow. In my first post on concurrency APIs, I called this "violating causality", and found that it was the root cause of many common, real-world issues in programs using asyncio and Twisted, like problems with backpressure, problems with shutting down properly, and so forth.

Go statements break automatic resource cleanup. Let's look again at that `with` statement example:

```
# Python
with open("my-file") as file_handle:
    ...
```

Before, we said that we were "guaranteed" that the file will be open while the `...` code is running, and then closed afterwards. But what if the `...` code spawns a background task? Then our guarantee is lost: the operations that *look* like they're inside the `with` block might actually keep running *after* the `with` block ends, and then crash because the file gets closed while they're still using it. And again, you can't tell from local inspection; to know if this is happening you have to go read the source code to all the functions called inside the `...` code.

If we want this code to work properly, we need to somehow keep track of any background tasks, and manually arrange for the file to be closed only when they're finished. It's doable – unless we're using some library that doesn't provide any way to get notified when the task is finished, which is distressingly common (e.g. because it doesn't expose any task handle that you can join on). But even in the best case, the unstructured control flow means the language can't help us. We're back to implementing resource cleanup by hand, like in the bad old days.

Go statements break error handling. Like we discussed above, modern languages provide powerful tools like exceptions to help us make sure that errors are detected and propagated to the right place. But these tools depend on having a reliable concept of "the current code's caller". As soon as you spawn a task or register a callback, that concept is broken. As a result, every mainstream concurrency framework I know of simply gives up. If an error occurs in a background task, and you don't handle it manually, then the runtime just... drops it on the floor and crosses its fingers that it wasn't too important. If you're lucky it might print something on the console. (The only other software I've used that thinks "print something and keep going" is a good error handling strategy is grotty old Fortran libraries, but here we are.) Even Rust – the language voted Most Obsessed With Threading Correctness by its high school class – is guilty of this. If a background thread panics, Rust discards the error and hopes for the best.

Of course you *can* handle errors properly in these systems, by carefully making sure to join every thread, or by building your own error propagation mechanism like errbacks in Twisted or Promise.catch in Javascript. But now you're writing an ad-hoc, fragile reimplementation of the features your language already has. You've lost useful stuff like "tracebacks" and "debuggers". All it takes is forgetting to call `Promise.catch` once and suddenly you're dropping serious errors on the floor without even realizing. And even if you do somehow solve all these problems, you'll still end up with two redundant systems for doing the same thing.

`go` statements: not even once

Just like `goto` was the obvious primitive for the first practical high-level languages, `go` was the obvious primitive for the first practical concurrency frameworks: it matches how the underlying schedulers actually work, and it's powerful enough to implement any other concurrent flow pattern. But again like `goto`, it breaks control flow abstractions, so that merely having it as an option in your language makes everything harder to use.

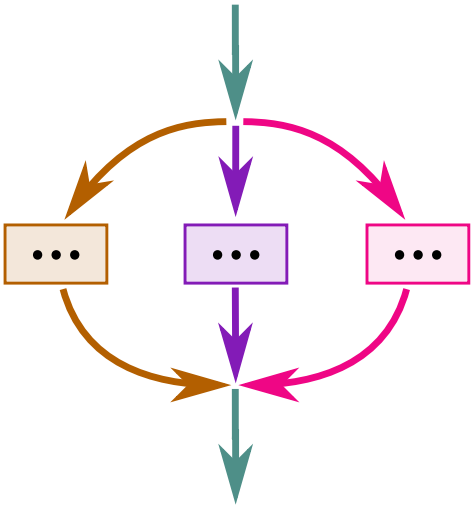
The good news, though, is that these problems can all be solved: Dijkstra showed us how! We need to:

- Find a replacement for `go` statements that has similar power, but follows the "black box rule",
- Build that new construct into our concurrency framework as a primitive, and don't include any form of `go` statement.

And that's what Trio did.

Nurseries: a structured replacement for `go` statements

Here's the core idea: every time our control splits into multiple concurrent paths, we want to make sure that they join up again. So for example, if we want to do three things at the same time, our control flow should look something like this:

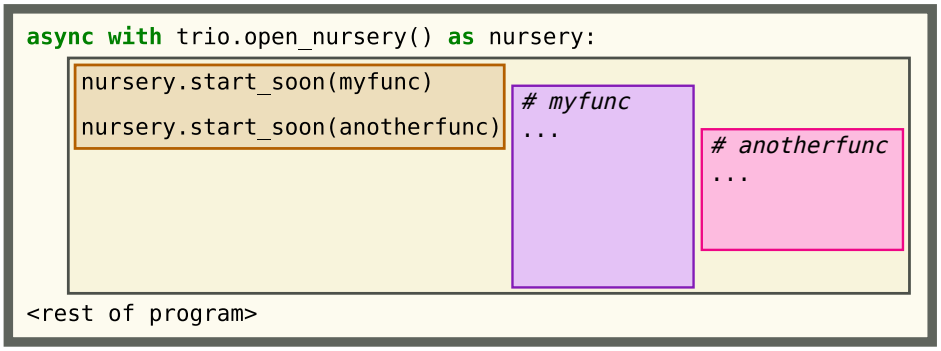


Notice that this has just one arrow going in the top and one coming out the bottom, so it follows Dijkstra's black box rule. Now, how can we turn this sketch into a concrete language construct? There are some existing constructs that meet this constraint, but (a) my proposal is slightly different than all the ones I'm aware of and has advantages over them (especially in the context of wanting to make this a standalone primitive), and (b) the concurrency literature is vast and complicated, and trying to pick apart all the history and tradeoffs would totally derail the argument, so I'm going to defer that to a separate post. Here, I'll just focus on explaining my solution. But please be aware that I'm not claiming to have like, invented the idea of concurrency or something, this draws inspiration from many sources, I'm standing on the shoulders of giants, etc. [3]

Anyway, here's how we're going to do it: first, we declare that a parent task cannot start any child tasks unless it first creates a place for the children to live: a *nursery*. It does this by opening a *nursery block*; in Trio, we do this using Python's `async with` syntax:

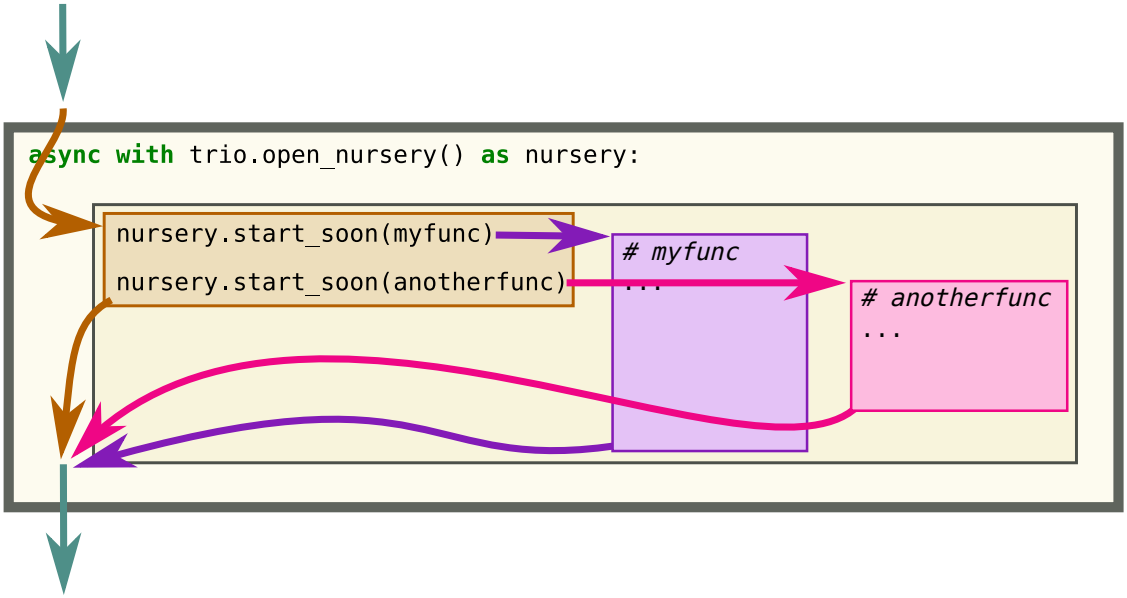
```
async with trio.open_nursery() as nursery:
    nursery.start_soon(myfunc)
    nursery.start_soon(anotherfunc)
<rest of program>
```

Opening a nursery block automatically creates an object representing this nursery, and the `as nursery` syntax assigns this object to the variable named `nursery`. Then we can use the nursery object's `start_soon` method to start concurrent tasks: in this case, one task calling the function `myfunc`, and another calling the function `anotherfunc`. Conceptually, these tasks execute *inside* the nursery block. In fact, it's often convenient to think of the code written inside the nursery block as being an initial task that's automatically started when the block is created.



Crucially, the nursery block doesn't exit until all the tasks inside it have exited – if the parent task reaches the end of the block before all the children are finished, then it pauses there and waits for them. The nursery automatically expands to hold the children.

Here's the control flow: you can see how it matches the basic pattern we showed at the beginning of this section:



This design has a number of consequences, not all of which are obvious. Let's think through some of them.

Nurseries preserve the function abstraction.

The fundamental problem with `go` statements is that when you call a function, you don't know whether it's going to spawn some background task that keeps running after it's finished. With nurseries, you don't have to worry about this: any function can open a nursery and run multiple concurrent tasks, but the function can't return until they've all finished. So when a function does return, you know it's really done.

Nurseries support dynamic task spawning.

Here's a simpler primitive that would also satisfy our flow control diagram above. It takes a list of thunks, and runs them all concurrently:

```
run_concurrently([myfunc, anotherfunc])
```

But the problem with this is that you have to know up front the complete list of tasks you're going to run, which isn't always true. For example, server programs generally have `accept` loops, that take incoming connections and start a new task to handle each of them. Here's a minimal `accept` loop in Trio:

```
async with trio.open_nursery() as nursery:
    while True:
        incoming_connection = await server_socket.accept()
        nursery.start_soon(connection_handler, incoming_connection)
```

With nurseries, this is trivial, but implementing it using `run_concurrently` would be *much* more awkward. And if you wanted to, it would be easy to implement `run_concurrently` on top of nurseries – but it's not really necessary, since in the simple cases `run_concurrently` can handle, the nursery notation is just as readable.

There is an escape.

The nursery object also gives us an escape hatch. What if you really do need to write a function that spawns a background task, where the background task outlives the function itself? Easy: pass the function a nursery object. There's no rule that only the code directly inside the `async with open_nursery()` block can call `nursery.start_soon` – so long as the nursery block remains open [4], then anyone who acquires a reference to the nursery object gets the capability of spawning tasks into that nursery. You can pass it in as a function argument, send it through a queue, whatever.

In practice, this means that you can write functions that "break the rules", but within limits:

- Since nursery objects have to be passed around explicitly, you can immediately identify which functions violate normal flow control by looking at their call sites, so local reasoning is still possible.
- Any tasks the function spawns are still bound by the lifetime of the nursery that was passed in.
- And the calling code can only pass in nursery objects that it itself has access to.

So this is still very different from the traditional model where any code can at any moment spawn a background task with unbounded lifetime.

One place this is useful is in the proof that nurseries have equivalent expressive power to `go` statements, but this post is already long enough so I'll leave that for another day.

You can define new types that quack like a nursery.

The standard nursery semantics provide a solid foundation, but sometimes you want something different. Perhaps you're envious of Erlang and its supervisors, and want to define a nursery-like class that handles exceptions by restarting the child task. That's totally possible, and to your users, it'll look just like a regular nursery:

```
async with my_supervisor_library.open_supervisor() as nursery_alike:
    nursery_alike.start_soon(...)
```

If you have a function that takes a nursery as an argument, then you can pass it one of these instead to control the error-handling policy for the tasks it spawns. Pretty nifty. But there is one subtlety here that pushes Trio towards different conventions than asyncio or some other libraries: it means that `start_soon` has to take a function, not a coroutine object or a `Future`. (You can call a function multiple times, but there's no way to restart a coroutine object or a `Future`.) I think this is the better convention anyway for a number of reasons (especially since Trio doesn't even have `Futures`!), but still, worth mentioning.

No, really, nurseries *always* wait for the tasks inside to exit.

It's also worth talking about how task cancellation and task joining interact, since there are some subtleties here that could – if handled incorrectly – break the nursery invariants.

In Trio, it's possible for code to receive a cancellation request at any time. After a cancellation is requested, then the next time the code executes a "checkpoint" operation (details), a `Cancelled` exception is raised. This means that there's a gap between when a cancellation is *requested* and when it actually *happens* – it might be a while before the task executes a checkpoint, and then after that the exception has to unwind the stack, run cleanup handlers, etc. When this happens, the nursery always waits for the full cleanup to happen. We *never* terminate a task without giving it a chance to run cleanup handlers, and we *never* leave a task to run unsupervised outside of the nursery, even if it's in the process of being cancelled.

Automatic resource cleanup works.

Because nurseries follow the black box rule, they make `with` blocks work again. There's no chance that, say, closing a file at the end of a `with` block will accidentally break a background task that's still using that file.

Automated error propagation works.

As noted above, in most concurrency systems, unhandled errors in background tasks are simply discarded. There's literally nothing else to do with them.

In Trio, since every task lives inside a nursery, and every nursery is part of a parent task, and parent tasks are required to wait for the tasks inside the nursery... we *do* have something we can do with unhandled errors. If a background task terminates with an exception, we can rethrow it in the parent task. The intuition here is that a nursery is something like a "concurrent call" primitive: we can think of our example above as calling `myfunc` and `anotherfunc` at the same time, so our call stack has become a tree. And exceptions propagate up this call tree towards the root, just like they propagate up a regular call stack.

There is one subtlety here though: when we re-raise an exception in the parent task, it will start propagating in the parent task. Generally, that means that the parent task will exit the nursery block. But we've already said that the parent task cannot leave the nursery block while there are still child tasks running. So what do we do?

The answer is that when an unhandled exception occurs in a child, Trio immediately cancels all the other tasks in the same nursery, and then waits for them to finish before re-raising the exception. The intuition here is that exceptions cause the stack to unwind, and if we want to unwind past a branch point in our stack tree, we need to unwind the other branches, by cancelling them.

This does mean though that if you want to implement nurseries in your language, you may need some kind of integration between the nursery code and your cancellation system. This might be tricky if you're using a language like C# or Golang where cancellation is usually managed through manual object passing and convention, or (even worse) one that doesn't have a generic cancellation mechanism.

A surprise benefit: removing `go` statements enables new features

Eliminating `goto` allowed previous language designers to make stronger assumptions about the structure of programs, which enabled new features like `with` blocks and exceptions; eliminating `go` statements has a similar effect. For example:

- Trio's cancellation system is easier to use and more reliable than competitors, because it can assume that tasks are nested in a regular tree structure; see Timeouts and cancellation for humans for a full discussion.
- Trio is the only Python concurrency library where control-C works the way Python developers expect (details). This would be impossible without nurseries providing a reliable mechanism for propagating exceptions.

Nurseries in practice

So that's the theory. How's it work in practice?

Well... that's an empirical question: you should try it and find out! But seriously, we just won't know for sure until lots of people have pounded on it. At this point I'm pretty confident that the foundation is sound, but maybe we'll realize we need to make some tweaks, like how the early structured programming advocates eventually backed off from eliminating `break` and `continue`.

And if you're an experienced concurrent programmer who's just learning Trio, then you should expect to find it a bit rocky at times. You'll have to learn new ways to do things – just like programmers in the 1970s found it challenging to learn how to write code without `goto`.

But of course, that's the point. As Knuth wrote (Knuth, 1974, p. 275):

> Probably the worst mistake any one can make with respect to the subject of **go to** statements is to assume that "structured programming" is achieved by writing programs as we always have and then eliminating the **go to**'s. Most **go to**'s shouldn't be there in the first place! What we really want is to conceive of our program in such a way that we rarely even *think* about **go to** statements, because the real need for them hardly ever arises. The language in which we express our ideas has a strong influence on our thought processes. Therefore, Dijkstra asks for more new language features – structures which encourage clear thinking – in order to avoid the **go to**'s temptations towards complications.

And so far, that's been my experience with using nurseries: they encourage clear thinking. They lead to designs that are more robust, easier to use, and just better all around. And the limitations actually make it easier to solve problems, because you spend less time being tempted towards unnecessary complications. Using Trio has, in a very real sense, taught me to be a better programmer.

For example, consider the Happy Eyeballs algorithm (RFC 8305), which is a simple concurrent algorithm for speeding up the establishment of TCP connections. Conceptually, the algorithm isn't complicated – you race several connection attempts against each other, with a staggered start to avoid overloading the network. But if you look at Twisted's best implementation, it's almost 600 lines of Python, and still has at least one logic bug. The equivalent in Trio is more than **15x** shorter. More importantly, using Trio I was able to write it in minutes instead of months, and I got the logic correct on my first try. I never could have done this in any other framework, even ones where I have much more experience. For more details, you can watch my talk at Pyninsula last month. Is this typical? Time will tell. But it's certainly promising.

Conclusion

The popular concurrency primitives – `go` statements, thread spawning functions, callbacks, futures, promises, ... they're all variants on `goto`, in theory and in practice. And not even the modern domesticated `goto`, but the old-testament fire-and-brimstone `goto`, that could leap across function boundaries. These primitives are dangerous even if we don't use them directly, because they undermine our ability to reason about control flow and compose complex systems out of abstract modular parts, and they interfere with useful language features like automatic resource cleanup and error propagation. Therefore, like `goto`, they have no place in a modern high-level language.

Nurseries provide a safe and convenient alternative that preserves the full power of your language, enables powerful new features (as demonstrated by Trio's cancellation scopes and control-C handling), and can produce dramatic improvements in readability, productivity, and correctness.

Unfortunately, to fully capture these benefits, we do need to remove the old primitives entirely, and this probably requires building new concurrency frameworks from scratch – just like eliminating `goto` required designing new languages. But as impressive as FLOW-MATIC was for its time, most of us are glad that we've upgraded to something better. I don't think we'll regret switching to nurseries either, and Trio demonstrates that this is a viable design for practical, general-purpose concurrency frameworks.

Comments

You can discuss this post on the Trio forum.

Acknowledgments

Many thanks to Graydon Hoare, Quentin Pradet, and Hynek Schlawack for comments on drafts of this post. Any remaining errors, of course, are all my fault.

Credits: Sample FLOW-MATIC code from this brochure (PDF), as preserved by the Computer History Museum. Wolves in Action, by i:am. photography / Martin Pannier, licensed under CC-BY-SA 2.0, cropped. French Bulldog Pet Dog by Daniel Borker, released under the CC0 public domain dedication.

Footnotes

[1] At least for a certain kind of person.

[2] And WebAssembly even demonstrates that it's possible and at least somewhat desirable have a low-level assembly language without `goto`: reference, rationale

[3] For those who can't possibly pay attention to the text without first knowing whether I'm aware of their favorite paper, my current list of topics to include in my review are: the "parallel composition" operator in Cooperating/Communicating Sequential Processes and Occam, the fork/join model, Erlang supervisors, Martin Sústrik's article on Structured concurrency and work on libdill, and crossbeam::scope / rayon::scope in Rust. [Edit: I've also been pointed to the highly relevant golang.org/x/sync/errgroup and github.com/oklog/run in Golang.] If I'm missing anything important, let me know.

[4] If you call `start_soon` *after* the nursery block has exited, then `start_soon` raises an error, and conversely, if it doesn't raise an error, then the nursery block is guaranteed to remain open until the task finishes. If you're implementing your own nursery system then you'll want to handle synchronization carefully here.

Posted by Nathaniel J. Smith Wed 25 April 2018